Inyección SQL

Diapositivas de apoyo: Inyección SQL y Los datos no dan instrucciones. Guion oral con ejemplo explicado; la demostración en vivo se reserva para la parte de laboratorio. Ensayar a 125-145 palabras por minuto.

De una búsqueda a una consulta

Continuando con el recorrido que acabamos de ver, nos concentraremos en los datos. Imaginemos un buscador de estudiantes. Escribimos Ana y esperamos que el programa encuentre el registro de Ana. Para hacerlo, el servidor puede consultar una base de datos mediante SQL. SQL es un lenguaje para pedir, agregar o modificar información organizada en tablas. No necesitamos aprender todo ese lenguaje para entender el ataque. Basta con reconocer que una consulta tiene una estructura y que el nombre buscado debería ser únicamente un valor dentro de esa estructura.

Una consulta parecida a seleccionar usuarios donde el nombre sea Ana expresa una intención limitada. El problema surge si el programa construye esa orden pegando directamente lo que alguien escribe. Concatenar significa unir fragmentos de texto. Es una operación normal y útil en muchos contextos, pero usarla para insertar entradas en una orden SQL puede mezclar dos papeles: lo que el desarrollador quiere ejecutar y lo que una persona aporta como dato. Cuando la entrada modifica la estructura de la consulta, hablamos de inyección SQL, o SQLi.

Podemos imaginar un formulario de papel con la instrucción buscar el expediente de esta persona. El espacio en blanco debería aceptar un nombre. Ahora imaginemos que alguien escribe algo que el encargado interpreta como una instrucción adicional: en realidad, entrégueme todos los expedientes. La analogía no describe exactamente el funcionamiento interno de la base de datos, pero ayuda a ver el error de diseño. La aplicación dejó que un dato participara en la definición de la orden. El ataque aprovecha esa confusión, no un poder especial de las palabras.

Funcionamiento y alcance

El proceso tiene tres pasos. Primero, una entrada controlada por el usuario llega al servidor, por ejemplo desde un campo de búsqueda. Segundo, el servidor la incorpora sin separación adecuada en una consulta. Tercero, la base de datos interpreta el resultado como una instrucción completa. Si la entrada altera la condición de búsqueda, pueden aparecer filas que no estaban previstas. El lugar de entrada no tiene que ser siempre una caja visible; también puede ser un parámetro de una dirección. Lo importante es cómo termina utilizándose el valor.

En el laboratorio veremos una condición que resulta verdadera para todos los registros de una tabla ficticia. La aplicación vulnerable devolverá dos usuarios en lugar de uno. Esto muestra una pérdida de control sobre la selección de datos. No significa que todas las inyecciones SQL permitan hacer exactamente lo mismo. El efecto depende de la consulta, del motor de base de datos, de la biblioteca utilizada y de los permisos disponibles. Una demostración pequeña permite entender la causa sin afirmar que reproduce todos los escenarios posibles.

Libreto de la comparación: Una búsqueda. Dos resultados.

Miremos las dos ventanas. A la izquierda, Ana busca su nombre y obtiene únicamente su registro. A la derecha no se introduce un nombre: aparece una cadena que agrega una condición a la consulta. No hace falta memorizarla; la parte importante es que expresa una condición verdadera. El programa vulnerable pega esa entrada en su orden SQL.

Ahora observemos el resultado después del clic. En la izquierda seguimos viendo solo a Ana. En la derecha aparecen Ana y Bruno, con sus correos. Así se ve el problema: una búsqueda que debía devolver un registro terminó mostrando información de otra persona. No cambiamos ni borramos datos; estamos observando una lectura no autorizada.

La siguiente diapositiva explica la prevención: enviar la estructura de la consulta y el valor por separado. La comparación animada muestra uso normal frente a abuso, no dos versiones de una defensa.

Riesgos para personas y organizaciones

El riesgo más fácil de visualizar es la lectura de información ajena. En nuestra universidad imaginaria, podrían aparecer expedientes que no corresponden al estudiante que consulta. En otro sistema podrían exponerse pedidos o datos de contacto. La información no necesita ser una contraseña para resultar sensible. Una lista de relaciones entre personas, actividades y horarios también puede causar daño. Por eso no deberíamos evaluar el problema solamente preguntando si alguien logra entrar como administrador. Ver datos sin permiso ya representa un incidente importante.

Otro posible impacto es la alteración o eliminación de registros, cuando la operación y los permisos lo permiten. Una nota incorrecta, un pedido modificado o un registro perdido afectan la integridad y pueden interrumpir el servicio. Algunas consultas de autenticación vulnerables también pueden permitir evadir una comprobación de acceso. Es importante decir puede, porque no todas las consultas son de inicio de sesión ni todas las cuentas tienen los mismos privilegios. La gravedad debe analizarse en el sistema concreto, sin minimizar el problema ni prometer un resultado universal.

La prevención principal

La medida central son las consultas parametrizadas. Parametrizar significa enviar la estructura de la consulta y sus valores por canales separados dentro de la biblioteca de acceso a datos. En la diapositiva aparece un signo de interrogación. En sqlite3, ese marcador indica dónde se utilizará un valor. El nombre se entrega aparte, como elemento de una secuencia. La biblioteca trata el contenido completo como dato. Si alguien introduce una cadena extraña, la búsqueda intenta encontrar esa cadena como nombre, sin convertirla en una nueva condición SQL.

Pensemos en un formulario con casilleros rígidos. La orden ya está definida y el dato entra en su casillero. No hace falta saber todas las variantes del ataque para aplicar este principio. Es más sólido que intentar adivinar y bloquear todas las cadenas sospechosas. Tampoco basta con eliminar comillas de cualquier texto. Esa decisión puede romper nombres legítimos y no ofrece una defensa general. La validación del formato sigue siendo útil, pero cumple otro propósito: comprobar que la entrada tenga el tipo y la forma que espera el negocio.

Reducir el daño y comprobar la solución

Otra capa es el mínimo privilegio. La cuenta utilizada por una función de lectura no necesita permisos para administrar toda la base de datos. Limitar permisos reduce las consecuencias de un error, aunque no corrige una consulta vulnerable. También conviene evitar respuestas que muestren detalles internos innecesarios y registrar errores para que el equipo pueda investigarlos. El registro debe ayudar a diagnosticar sin guardar secretos. Cada capa tiene una función: prevenir la confusión, restringir el alcance y facilitar la detección.

Para verificar una corrección, no basta con observar que una cadena dejó de funcionar. Debemos comprobar que las búsquedas normales siguen funcionando y revisar que el código envía los valores por separado. En nuestro ejemplo, Ana debe devolver su registro; la cadena de prueba en la versión segura no debería devolver usuarios. Si la página simplemente se rompe, no hemos demostrado una buena solución. La seguridad también necesita conservar el comportamiento legítimo. Podemos recordar la prevención con una frase sencilla: la consulta decide la operación y la entrada aporta solamente el valor.

Con esto cerramos el ataque a la base de datos. Ahora cambiaremos de escenario: en lugar de una entrada interpretada por SQL, veremos contenido interpretado por el navegador. La pregunta vuelve a ser parecida: ¿se está mostrando un dato o se está ejecutando una instrucción? Mi compañero explicará XSS, sus variantes y las consecuencias para quien visita una página. Mantengamos esa comparación, porque nos permitirá conectar los temas sin confundir sus soluciones.

Apoyo para comprender y ensayar

SQL se puede pronunciar letra por letra. Fila equivale a registro; columna equivale a atributo. Parametrizar no es cifrar. Un ORM ayuda a construir consultas, pero sus operaciones de SQL directo también deben usarse correctamente. Si preguntan por contraseñas, explicar que su almacenamiento seguro es un tema adicional: no evita que otras consultas tengan SQLi. No ejecutar el laboratorio en esta parte para evitar repetir tiempo y contenido.

Indicaciones para presentar esta parte

Desde Inyección SQL, avanzar una vez para abrir Una búsqueda. Dos resultados. Esperar la escritura, el clic y las dos tablas; narrar primero la izquierda y después la derecha. No usar mouse, Enter, formularios ni selectores: todo es animación. Avanzar después a la prevención. Volver a entrar reinicia la secuencia.

Diccionario de términos

SQL: Lenguaje para consultar y modificar bases de datos relacionales.

Consulta: Instrucción que pide información o una operación a la base de datos.

Concatenación: Unión de fragmentos de texto; es peligrosa si mezcla una entrada con instrucciones SQL.

SQLi: Inyección SQL: una entrada altera la estructura prevista de una consulta.

Parámetro: Valor entregado a la biblioteca por separado de la estructura SQL.

Fila y columna: Una fila representa un registro; una columna, una propiedad del registro.

Mínimo privilegio: Otorgar únicamente los permisos necesarios para una tarea.

ORM: Herramienta que permite trabajar con datos mediante objetos; su SQL directo también exige cuidado.